

Skill-12

```
himasree@LAPTOP-41COT5AA:~/OSLab$ ./multi_pipeline
=====
MULTI PIPELINE DEMONSTRATOR
=====

Test 1:
ls | grep .c | wc -l

Created Pipe 1: read=3 write=4
Created Pipe 2: read=5 write=6
101

Test 2:
echo Hello World | grep Hello | wc -w

Created Pipe 1: read=3 write=4
Created Pipe 2: read=5 write=6
2

Test 3 - Long Pipeline:
ls | grep .c | sort | uniq | wc -l

Created Pipe 1: read=3 write=4
Created Pipe 2: read=5 write=6
Created Pipe 3: read=7 write=8
Created Pipe 4: read=9 write=10
101

All pipeline tests completed.
himasree@LAPTOP-41COT5AA:~/OSLab$ |
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat multi_pipeline.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 50
#define MAX_INPUT 1024

/* Parse command into arguments */
void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}
```

```

/* Execute multiple connected commands */
void execute_pipeline(char *commands[], int command_count)
{
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    /*
     * Create N-1 pipes for N commands
     */
    for (int i = 0; i < command_count - 1; i++) {

        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            exit(EXIT_FAILURE);
        }

        printf("Created Pipe %d: read=%d write=%d\n",
               i + 1,
               pipes[i][0],
               pipes[i][1]);
    }

    /*
     * Create one process for each command
     */
    for (int i = 0; i < command_count; i++) {

        pids[i] = fork();

        if (pids[i] < 0) {

```

```

        char command_copy[MAX_INPUT];
        char *args[MAX_ARGS];

        strcpy(command_copy, commands[i]);

        parse_command(command_copy, args);

        /*
         * If this is not the first command,
         * take input from previous pipe.
         */
        if (i > 0) {

            if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1) {
                perror("dup2 input");
                exit(EXIT_FAILURE);
            }
        }

        /*
         * If this is not the last command,
         * send output to next pipe.
         */
        if (i < command_count - 1) {

            if (dup2(pipes[i][1], STDOUT_FILENO) == -1) {
                perror("dup2 output");
                exit(EXIT_FAILURE);
            }
        }
    }
}

```

```

        perror("dup2 output");
        exit(EXIT_FAILURE);
    }
}

/*
 * Close all pipe descriptors in child
 */
for (int j = 0; j < command_count - 1; j++) {
    close(pipes[j][0]);
    close(pipes[j][1]);
}

/* Execute command */
if (args[0] == NULL) {
    exit(EXIT_FAILURE);
}

execvp(args[0], args);

perror("execvp");
exit(EXIT_FAILURE);
}
}

/*
 * Parent closes all pipe descriptors
 */
for (int i = 0; i < command_count - 1; i++) {
    close(pipes[i][0]);

```

```

printf("=====\n");
printf("      MULTI PIPELINE DEMONSTRATOR\n");
printf("=====\n\n");

/*
 * TEST 1
 * ls | grep .c | wc -l
 */
printf("Test 1:\n");
printf("ls | grep .c | wc -l\n\n");

char *pipeline1[] = {
    "ls",
    "grep .c",
    "wc -l"
};

execute_pipeline(pipeline1, 3);

/*
 * TEST 2
 * echo Hello World | grep Hello | wc -w
 */
printf("\nTest 2:\n");
printf("echo Hello World | grep Hello | wc -w\n\n");

char *pipeline2[] = {
    "echo Hello World",
    "grep Hello",

```

```
        "grep Hello",
        "wc -w"
    };

    execute_pipeline(pipeline2, 3);

    /*
     * TEST 3
     * Long pipeline
     */
    printf("\nTest 3 - Long Pipeline:\n");
    printf("ls | grep .c | sort | uniq | wc -l\n\n");

    char *pipeline3[] = {
        "ls",
        "grep .c",
        "sort",
        "uniq",
        "wc -l"
    };

    execute_pipeline(pipeline3, 5);

    printf("\nAll pipeline tests completed.\n");

    return 0;
}
himasree@APT0P-#1COT5AA:~/OSLab$ |
```